

SetMyCareer – Complete API Reference

Every endpoint in the system, in one place.

Version	1.0 — 18 July 2026
Companion to	SMC_SYSTEM_ARCHITECTURE.md (information architecture + the C# implementation spec). This file is the flat catalogue; that file is the design.
Audience	Backend developers and integrators. Readable without having used the product.

The platform runs on **four API layers**. This document lists **every endpoint in all four**, what it does, its method, and — where it changes data — whether it is a write and how it is guarded.

#	Layer	Base URL	Endpoints	Status
1	New C# API — the system to build	<code>https://api.setmycareer.com/api/</code> (new controllers)	47	Specified, ready to implement
2	Live SMC backend — already yours	<code>https://api.setmycareer.com/api/</code>	53	Live in production
3	Vercel serverless — AI, payments, cloud	<code>https://setmycareer-counselor.vercel.app/api/</code>	11	Live in production
4	LangGraph service — the report brain	Railway (<code>LANGGRAPH_BASE_URL</code>)	30	Live; C# orchestrates it
Total			141	

Conventions (all layers unless noted): base `.../api/`, Content-Type: `application/json` (file upload = `multipart/form-data`), numeric ids sent as **strings**, success envelope `{ "success": true, "data": ... } / { "success": false, "message": "..." }`. CORS allows the two app origins for `GET`, `POST`, `PUT`, `DELETE`, `OPTIONS` + `Authorization`.

The non-destructive rule (Layer 1): no `DELETE` verbs; clients are soft-archived only; `TestResults`, `CreditLedger`, `AuditEvents`, `SynthesisJobs` are append-only; prices come from the server; no AI/payment secret ever reaches a browser. See the architecture doc §2.0.

Layer 1 – New C# API (the system to build)

ASP.NET Core 8. Roles: `Client`, `Counsellor`, `Admin` (JWT bearer from the existing logins). Full entity/DbContext/controller code is in the architecture doc §2.2–§2.5. **47 endpoints, zero DELETE verbs.**

A. Accounts – AccountsController (closes the local-only sign-up gap)

Method	Route	Role	Purpose
POST	Accounts/Create	anon	Create a real backend client user + seed ClientProfile ; returns {userId, jwt} . Idempotent on email.
POST	Accounts/SendMagicLink	anon	Email a one-time sign-in link (makes a created account work cross-device).
GET	Accounts/Me	Client	Portal boot payload: profile + credits + revoke flag in one call.

B. Profile – ProfileController

Method	Route	Role	Purpose
GET	Profile/Get	Client (own) / Counsellor (caseload) / Admin	The intake record + completeness %.
PUT	Profile/Update	Client	Create-or-update own profile; server recomputes CompletedAtUtc . Gates read this.

C. Tests – TestsController (append-only – a retake is a new row)

Method	Route	Role	Purpose
POST	Tests/Submit	Client	New TestResult row (scores + full payload + raw answers). Auto-enqueues synthesis when the battery of 3 completes.
GET	Tests/ListMine	Client	All own results, newest first (history feeds progress-over-time).
GET	Tests/ListForClient?userId=	Counsellor / Admin	Same shape, for the client 360°.

D. Sessions — `SessionsController`

Method	Route	Role	Purpose
POST	<code>Sessions/Request</code>	Client	Booking → status <code>requested</code> ; auto-assigns a counsellor (expertise × availability × profile) and stores the reason.
GET	<code>Sessions/MyBookings</code>	Client	Own bookings + note count + transcript flag.
GET	<code>Sessions/Caseload</code>	Counsellor	Own upcoming + past bookings.
POST	<code>Sessions/Confirm</code>	Counsellor / Admin	<code>requested</code> → <code>confirmed</code> (state machine; invalid transitions rejected).
POST	<code>Sessions/Cancel</code>	Counsellor / Admin	any → <code>canceled</code> .
POST	<code>Sessions/Complete</code>	Counsellor / Admin	<code>confirmed</code> → <code>completed</code> .
POST	<code>Sessions/Reschedule</code>	Counsellor / Admin	New time → status back to <code>requested</code> for client re-confirm; audited.
POST	<code>Sessions/AddNote</code>	Counsellor	Timestamped session note (append-only).
GET	<code>Sessions/Notes?bookingId=</code>	Client (own) / Counsellor / Admin	The recap notes.
GET	<code>Sessions/Transcript?bookingId=</code>	Client (own) / Counsellor / Admin	The transcript (feeds portal “Read full screen” + “Export” + Compass hand-off).
PUT	<code>Sessions/SaveTranscript</code>	Counsellor	Upsert transcript until the booking is <code>completed</code> , then frozen (409).

E. Synthesis — `SynthesisController` (the LangGraph bridge, §2.6)

Method	Route	Role	Purpose
POST	<code>Synthesis/Enqueue</code>	Client / Counsellor / Admin	Create a <code>SynthesisJob</code> snapshotting profile + latest 3 results; dedupes pending. Calls Layer 4.
GET	<code>Synthesis/Status?jobId=</code>	same	<code>queued</code> / <code>running</code> / <code>done</code> / <code>failed</code> + error.
GET	<code>Synthesis/Result?jobId=</code>	same	The stored counsellor report / blueprint.

Method	Route	Role	Purpose
GET	Synthesis/Latest?userId=	same	Newest done job — what Reports hub + 360° render.

F. Admin — AdminController

Method	Route	Role	Purpose
GET	Admin/AllUpcomingBookings	Admin	Every booking, nested client↔counsellor, with assignment reason.
POST	Admin/RevokePortalAccess	Admin	Set/clear the portal-access flag (portal signs a revoked client out on boot; cross-device).
POST	Admin/ArchiveClient	Admin	Soft -archive only — the sole “remove client”. Audited, reversible.
GET	Admin/AuditTrail	Admin	Filter every mutation by entity / actor / date.
GET	Admin/Calendars	Admin	Merged bookings + counsellor blocks, per counsellor per day.

G. Counsellor — CounsellorController

Method	Route	Role	Purpose
GET	Counsellor/Agreement	Counsellor	Read the service-agreement acceptance state.
POST	Counsellor/Agreement	Counsellor	Accept T&C + DPDP confidentiality + conduct (all three required). Server gates Confirm / AddNote on this.
GET	Counsellor/Blocks	Counsellor	Own calendar blocks (unavailability).
POST	Counsellor/Blocks	Counsellor	Create a block; feeds auto-assignment + admin calendars.
POST	Counsellor/CancelBlock	Counsellor	Archive a block (never hard-deleted).

H. App data, messaging, guest links, checkout

Ports of the interim cloud store + guest flow + payments.

Method	Route	Role	Purpose
GET	State/GetAll? app=&userId=	any (own)	Per-user app state (bookings, calendar, wallet, drafts).
POST	State/Set	any (own)	Upsert one key.
POST	State/Remove	any (own)	Tombstone a key (history kept).
GET	Chats/List? app=&userId=	any (own)	AI chat threads.
POST	Chats/Upsert	any (own)	Save a thread; cap 50 → archive oldest (never delete).
POST	Chats/Archive	any (own)	Archive a thread.
GET	Messages/Thread? threadKey=	Client / Counsellor	Client ↔ counsellor thread.
POST	Messages/Send	Client / Counsellor	Send a message (+ optional attachment).
POST	GuestLinks/Create	Admin	Mint a <code>/t/<token></code> guest test link (full battery or <code>?only=</code>).
GET	GuestLinks/Get? token=	anon	Validate a token + return its mode.
POST	GuestLinks/SubmitResult	anon	Store a guest test result (append-only).
POST	GuestLinks/Deactivate	Admin	Deactivate a link (never deleted).
POST	Checkout/Create	Client	Create a Razorpay order — price from the server catalog , never the client.
POST	Checkout/Verify	Client	HMAC-verify <code>order_id payment_id</code> ; only a valid signature writes <code>Purchase(paid)</code> + credit grants.

Layer 2 – Live SMC backend (already in production)

<https://api.setmycareer.com/api/> . **53 endpoints**. Reads are open (no token); **19 writes are gated** by `VITE_SMC_WRITES_ENABLED` (currently ON). Numeric ids as strings. These are consumed by the app today and stay

as-is.

Auth (8) – not gated (plain POST)

Method	Endpoint	Purpose
POST	Login/AdminLogin	Admin + counsellor staff sign-in (array response, len>0 = ok).
POST	Login/NavigatorLogin	Counsellor sign-in — keys on email/mobile; returns record even when disabled so the app gates on <code>isActive</code> .
POST	User/SendOtp	Client OTP start (phone or email).
POST	User/ResendOtp	Client OTP resend.
POST	User/SendOtpRegister	OTP for new-account registration.
POST	User/LoginWithOtp	Verify OTP → UserDetail (falls back to UserView).
POST	User/LoginWithPassword	Email/mobile + password client sign-in.
POST	User/SigninWithGoogle	Google sign-in bridge.

Reads – dashboards, roster, caseload (25) – open

Method	Endpoint	Purpose
GET	Admin/GetStatistics	Admin dashboard stats.
GET	NavigatorList/getAllNavigator	Full counsellor roster (~81); also classifies a login as counsellor.
GET	NavigatorList	Navigator dropdown list.
POST	NavigatorList/GetNavigatorListByPackageId	Counsellors available for a package + mode + location.
POST	NavigatorDetail	One navigator's detail by id.
POST	NavigatorDetail/GetNavigatorStats	Counsellor top stats (ClientCount / SessionCount / Rating / Earning).
POST	SoldService/getAllclientbysession	Admin all-clients-by-session rows.
POST	SoldService/getclientbyadmin	Admin client rows (full CLIENT_FILTER object).

Method	Endpoint	Purpose
POST	<code>SoldService/getClientbynaviId</code>	A navigator's entire caseload (heavy, 90s timeout, cached).
POST	<code>SoldService/getClientbynaviIdNavi</code>	Navigator dashboard upcoming-sessions feed (lighter).
POST	<code>SoldService/getSessionAllAdmin</code>	All admin session rows (~12k).
POST	<code>SoldService/getSessionAll</code>	One user's sessions (client/counsellor timeline).
POST	<code>UserView</code>	Full user profile by userId/mobile/email; <code>{userId:0}</code> = entire roster (~1 MB).
GET	<code>UserView/GetPurchasePackages/{userId}</code>	A user's purchased packages.
GET	<code>Reports/getAllReportsForUser/{userId}</code>	A user's uploaded report list.
POST	<code>SoldService/getCommentsAllbyNavi</code>	Session notes/comments for a user.
POST	<code>SoldService/getReviewbyid</code>	A user's reviews.
POST	<code>SoldService/primaryFormCheck</code>	Whether the client filled the primary intake form.
POST	<code>SoldService/getTestbyId</code>	A user's 4 test-status strings.
GET	<code>SoldService/GetAdminAbilitySummaryData</code>	Psychometric ability summary (~978 students, sa/cl/ra/ca/va2/na/ma).
GET	<code>RecommendedService/getRecommendedServices/{userId}</code>	Counsellor-recommended services for a client.
POST	<code>SoldService/getAllPackages</code>	Full package catalogue (~123); doubles as the connectivity probe.
GET	<code>Admission/GetAdmissionData/{userId}</code>	Saved admission-assistance preferences.
POST	<code>SoldService/getCareerExplorerQuestions</code>	Career Explorer Q&A for a service.
POST	<code>Calendar/ListAllEvents</code>	Navigator's Zoho calendar events (≤31-day window).
GET	<code>Calendar/GetEventDetails/{calendarId}/{eventId}</code>	One Zoho event's detail.

Writes – gated by `VITE_SMC_WRITES_ENABLED` (19)

Method	Endpoint	Purpose
POST	<code>SoldService/addClientServiceFromAdmin</code>	Sell a package to a client from admin.
POST	<code>SoldService/ModifysessionsStatus</code>	Change a session's status (+ full-session modify).

Method	Endpoint	Purpose
POST	<code>SoldService/deletesessions</code>	Soft -delete a session (<code>session_status: Deleted</code>).
POST	<code>NavigatorDetail/AddNavigator</code>	Create a counsellor account (name/email/password).
POST	<code>SoldService/updateNavigatorDetail</code>	Update a navigator profile.
POST	<code>NavigatorDetail/EnableNavigator/{email}</code>	Re-enable a navigator.
POST	<code>NavigatorDetail/disableNavigator/{email}</code>	Disable a navigator.
POST	<code>Calendar/changeUserNavigator</code>	Reassign a client's counsellor.
POST	<code>SoldService/commentinsert</code>	Add a session note (date DD/MM/YYYY).
POST	<code>UserUpdate/UserUpdateCategory</code>	Change a user's category (keyed on mobile).
POST	<code>RecommendedService/addRecommendedService</code>	Save a recommended service for a client.
POST	<code>Reports/uploadReport</code>	Multipart report PDF upload (user_id, report_name, file).
POST	<code>SoldService/postRecommendation</code>	Multipart recommendation-report upload.
POST	<code>SoldService/saveCareerExplorerAnswers</code>	Save navigator-entered Career Explorer answers.
POST	<code>Calendar/AddRecurringEvent</code>	Create a Zoho calendar event.
POST	<code>Calendar/UpdateEvent</code>	Update a Zoho event (etag/recurrence aware).
POST	<code>Calendar/DeleteEvent</code>	Delete a Zoho event.
POST	<code>Calendar/UploadAttachment</code>	Multipart calendar attachment upload.

`Payment/CreateOrder` exists in the endpoint map but is **permanently blocked** in the app (a stub that always throws) — real orders go through the Vercel `/api/razorpay` route (Layer 3) instead.

Layer 3 – Vercel serverless (AI, payments, cloud, media)

`https://setmycareer-counselor.vercel.app/api/` . **11 functions**. These hold the AI/payment/cloud secrets server-side; the browser never sees a key.

Route	Run time	Methods	Purpose	Keys used
<code>/api/assistent</code>	Edge	POST, OPTIONS	Compass chat — Groq → OpenRouter failover, audience-scoped personas (client / counsellor / admin), grounded in the user's own data.	<code>GROQ_API_KEY</code> , <code>OPENROUTER_API_KEY</code>

Route	Run time	Methods	Purpose	Keys used
/api/clo ud	Edge	POST	Interim per-user store (Supabase): {kind:"state" "chats", op:...} . Layer 1 §H replaces this.	SUPABASE_URL , SUPABASE_KEY
/api/con solidate	Edge	POST	Guest-battery synthesis into one on-screen report.	GEMINI_API_KEY
/api/fit -report	Edge	POST	Marketing fit-test → AI package-fit report.	GR0Q_API_KEY
/api/liv ekit- token	Edge	POST	Mint a LiveKit room token for a session/call.	LIVEKIT_URL , LIVEKIT_API_KEY , LIVEKIT_API_SECRET
/api/mar keting	Edge	POST	Google-Ads spend/metrics for the admin marketing screen.	GOOGLE_ADS_*
/api/not es	Edge	POST	Transcript/notes helper (session recap processing).	GR0Q_API_KEY
/api/pro viders	Nod e	GET, POST	Real API-usage/provider data for the admin API screen.	(reads platform state)
/api/raz orpay	Nod e	POST, OPTIO NS	Create order + HMAC-verify payment. Node runtime is required (Razorpay 406s on Edge).	RAZORPAY_KEY_ID , RAZORPAY_KEY_SECRET
/api/rep ort	Edge	POST	Counsellor career-report generation (McKinsey-grade).	GR0Q_API_KEY / OPENROUTER_API_KEY
/api/tra nscribe	Edge	POST	Speech-to-text for live session transcripts.	GR0Q_API_KEY (Whisper)

Layer 4 – LangGraph service (the report brain)

FastAPI on Railway (`uvicorn app.main:app`), reached via `LANGGRAPH_BASE_URL` . **30 endpoints**. The 7-agent career graph (`labor_retriever` → `career_metrics` → `evidence_verifier` → `behavioral_scientist` → `contradiction_agent` → `synthesis` → `counsellor_report`) lives here. The C# `SynthesisController` calls the two report endpoints; the rest support meetings, inventories, and the wellbeing bridge. AI keys (`GR0Q_API_KEY` , `OPENROUTER_API_KEY` , `LLM_MODEL` , `STT_MODEL`) live on **this** service.

Core report + analysis

Method	Endpoint	Purpose
GET	/api/health	Liveness.
POST	/api/career/report/specialised	The one the C# API calls — full blueprint from {career_profile, transcript?, mh_context?}.
POST	/api/career/report	Base career report.
POST	/api/career/analyze	Career analysis (non-streaming).
POST	/api/career/analyze/stream	Same, SSE updates stream for live UIs.
POST	/api/career	Career pipeline entry.
POST	/api/analyze	Generic analysis.
POST	/api/analyze/stream	Generic analysis, SSE.
POST	/api/chat	Grounded chat over the career brain.
POST	/api/insights	Insight extraction.
POST	/api/summarize	Summarisation.

Inventories, labor, wellbeing

Method	Endpoint	Purpose
GET	/api/inventories	List available psychometric inventories.
POST	/api/inventories/score	Score an inventory submission.
GET	/api/labor/outlook	BLS/ESCO labour-market outlook data.
POST	/api/bridge/wellbeing	Push a wellbeing-context bridge record.
GET	/api/bridge/wellbeing/{user_id}	Read a user's wellbeing context.

Transcription + meetings

Method	Endpoint	Purpose
POST	/api/transcribe	Server-side transcription.

Method	Endpoint	Purpose
POST	/api/diarize	Speaker diarization.
POST	/api/meetings/bot	Start a meeting-recording bot.
GET	/api/meetings/bot/{bot_id}	Bot status.
POST	/api/meetings/webhook	Meeting-provider webhook sink.
POST	/api/counselor/sessions	Record a counsellor session.
GET	/api/counselor/sessions/{client_id}	A client's recorded sessions.

Integrations + device pairing

Method	Endpoint	Purpose
GET	/api/integrations/status	Connected-integration status.
GET	/api/integrations/{provider}/connect	Begin an OAuth connect.
GET	/api/integrations/{provider}/callback	OAuth callback.
POST	/api/integrations/import	Import from a connected provider.
POST	/api/pair/create	Create a device-pairing code.
POST	/api/pair/join	Join via a pairing code.
GET	/api/pair/status/{code}	Pairing status.

Environment keys – complete index (names only, never values)

Location	Keys
C# API (new)	SMC_DB_CONNECTION , SMC_JWT_SECRET , RAZORPAY_KEY_ID , RAZORPAY_KEY_SECRET , LANGGRAPH_BASE_URL , LIVEKIT_URL , LIVEKIT_API_KEY , LIVEKIT_API_SECRET
LangGraph service (Railway)	GROQ_API_KEY , LLM_MODEL , OPENROUTER_API_KEY , OPENROUTER_MODEL , STT_MODEL , GOOGLE_CLIENT_ID , GOOGLE_CLIENT_SECRET , ZOHO_CLIENT_ID , PORT

Location	Keys
Vercel serverless	GEMINI_API_KEY , GOOGLE_GENERATIVE_AI_API_KEY , GROQ_API_KEY , OPENROUTER_API_KEY , LIVEKIT_URL , LIVEKIT_API_KEY , LIVEKIT_API_SECRET , RAZORPAY_KEY_ID , RAZORPAY_KEY_SECRET , SUPABASE_URL , SUPABASE_KEY (retired at Phase 3), GOOGLE_ADS_CLIENT_ID , GOOGLE_ADS_CLIENT_SECRET , GOOGLE_ADS_DEVELOPER_TOKEN , GOOGLE_ADS_CUSTOMER_ID , GOOGLE_ADS_LOGIN_CUSTOMER_ID , GOOGLE_ADS_REFRESH_TOKEN , GOOGLE_ADS_CURRENCY , GOOGLE_API_KEY
Frontend (public by design)	VITE_SMC_WRITES_ENABLED , Razorpay publishable key id only

Razorpay runs on **TEST** keys today; switching to LIVE is a deliberate, separate change (architecture doc §2.10 Phase 3).

Coverage checklist – is every product touchpoint served?

Touchpoint	Endpoint(s)
Client sign-up / boot	Accounts/Create , Accounts/SendMagicLink , Accounts/Me (L1) + live OTP/password auth (L2)
Profile intake + gates	Profile/Upsert / Profile/Get (L1)
Take any of the 4 tests	Tests/Submit (L1)
Reports hub + counsellor/admin 360°	Tests/ListForClient , Synthesis/Latest (L1)
Consolidated career report	Synthesis/* (L1) → /api/career/report/specialised (L4)
Book a session + auto-assign	Sessions/Request (L1)
Session recaps / transcripts / Compass hand-off	Sessions/Notes , Sessions/Transcript (L1)
Counsellor confirm / cancel / notes	Sessions/Confirm Cancel AddNote SaveTranscript (L1)
Counsellor agreement + calendar blocks	Counsellor/Agreement , Counsellor/Blocks (L1)
Admin oversight / reschedule / revoke / archive	Admin/* , Sessions/Reschedule (L1)
Compass chat (all doors)	Chats/* (L1) + /api/assistant (L3)
Client ↔ counsellor messaging	Messages/* (L1)

Touchpoint	Endpoint(s)
Package & credits / checkout	Checkout/Create + Checkout/Verify (L1) + /api/razorpay (L3)
Guest test links (/t/...)	GuestLinks/* (L1) + /api/consolidate (L4-adjacent)
Live video/voice calls	/api/livekit-token (L3)
Document uploads (mark-sheets etc.)	Reports/uploadReport (L2 — already yours)
Package catalogue / roster / live sessions	Layer 2 reads (already yours)
Marketing spend, API-usage screens	/api/marketing , /api/providers (L3)

Every user-facing action maps to a real endpoint. Nothing is left without a home.